

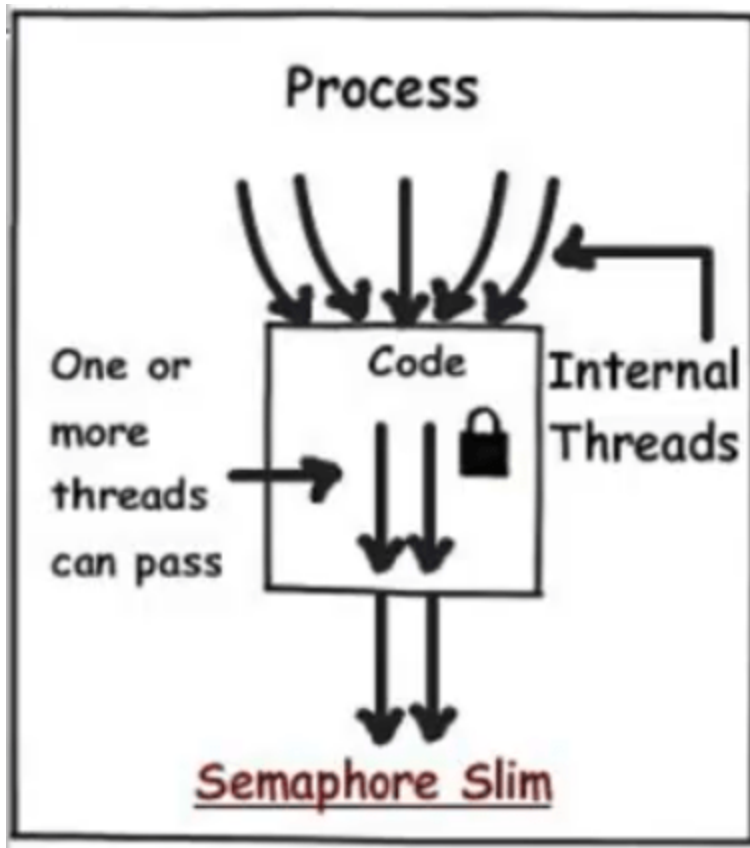
SemaphoreSlim Class in C# with Examples

In this article, I am going to discuss **How to Implement Thread Synchronization using SemaphoreSlim Class in C#** with Examples. Please read our previous article where we discussed [How to Implement Thread Synchronization using Semaphore Class in C#](#) with Examples. The SemaphoreSlim Class represents a lightweight alternative to Semaphore that limits the number of threads that can access a resource or pool of resources concurrently.

Why do we need SemaphoreSlim as we already have Lock, Monitor, Mutex, and Semaphore in C#?

Like [Lock](#), [Monitor](#), [Mutex](#), and [Semaphore](#), the SemaphoreSlim class in C# is also used to provide thread safety. The Lock and Monitors are basically used to provide thread safety for Internal Threads i.e. the threads generated by the application itself. On the other hand, Mutex and Semaphore ensure thread safety for threads that are generated by the external applications i.e. External Threads. Using Mutex, only one external thread can access our application code at any given point in time. And, if we want more control over the number of external threads that can access our application code, then can use Semaphore in C#.

Using Lock and Monitor, only one internal thread can access our application code at any given point in time. But, if we want more control over the number of internal threads that can access our application code, then we need to use SemaphoreSlim class in C#. For a better understanding, please have a look at the below image.



What is SemaphoreSlim class in C#?

The SemaphoreSlim Class in C# is recommended for synchronization within a single app. A lightweight semaphore controls access to a pool of resources that is local to your application. It represents a lightweight alternative to Semaphore that limits the number of threads that can access a resource or pool of resources concurrently.

Constructors, and Methods of SemaphoreSlim Class in C#:

Let us understand the different Constructors, and Methods of SemaphoreSlim Class in C#. If you right-click on the SemaphoreSlim class and select go to definition, then you will see the following class definition. Its a class and it implements the IDisposable interface.

```
namespace System.Threading
{
    ...public class SemaphoreSlim : IDisposable
    {
        ...public SemaphoreSlim(int initialCount);
        ...public SemaphoreSlim(int initialCount, int maxCount);

        ...public int CurrentCount { get; }
        ...public WaitHandle AvailableWaitHandle { get; }

        ...public void Dispose();
        ...public int Release();
        ...public int Release(int releaseCount);
        ...public bool Wait(TimeSpan timeout);
        ...public bool Wait(TimeSpan timeout, CancellationToken cancellationToken);
        ...public bool Wait(int millisecondsTimeout);
        ...public bool Wait(int millisecondsTimeout, CancellationToken cancellationToken);
        ...public void Wait();
        ...public void Wait(CancellationToken cancellationToken);
        ...public Task WaitAsync();
        ...public Task<bool> WaitAsync(int millisecondsTimeout);
        ...public Task<bool> WaitAsync(TimeSpan timeout);
        ...public Task<bool> WaitAsync(TimeSpan timeout, CancellationToken cancellationToken);
        ...public Task<bool> WaitAsync(int millisecondsTimeout, CancellationToken cancellationToken);
        ...public Task WaitAsync(CancellationToken cancellationToken);
        ...protected virtual void Dispose(bool disposing);
    }
}
```

Constructors of SemaphoreSlim Class in C#:

The SemaphoreSlim Class in C# provides the following two constructors that we can use to create an instance of the SemaphoreSlim class.

1. **SemaphoreSlim(int initialCount):** It initializes a new instance of the SemaphoreSlim class, specifying the initial number of requests that can be granted concurrently. Here, the parameter initialCount specifies the initial number of requests for the semaphore that can be granted concurrently. It will throw ArgumentOutOfRangeException if the initialCount is less than 0.
2. **SemaphoreSlim(int initialCount, int maxCount):** It initializes a new instance of the SemaphoreSlim class, specifying the initial and maximum number of requests that can be granted concurrently. Here, the parameter initialCount specifies the initial number of requests for the semaphore that can be granted concurrently. And the parameter maxCount specifies the maximum number of requests for the semaphore that can be granted concurrently. It will throw ArgumentOutOfRangeException if

initialCount is less than 0, or initialCount is greater than maxCount, or maxCount is equal to or less than 0.

Methods of SemaphoreSlim Class in C#:

The SemaphoreSlim Class in C# provides the following methods.

Wait Method:

There are multiple overloaded versions of the Wait method available in SemaphoreSlim Class. They are as follows:

1. **Wait():** It blocks the current thread until it can enter the System.Threading.SemaphoreSlim.
2. **Wait(TimeSpan timeout):** It blocks the current thread until it can enter the SemaphoreSlim, using a TimeSpan to specify the timeout. It returns true if the current thread successfully entered the SemaphoreSlim; otherwise, false.
3. **Wait(CancellationToken cancellationToken):** It blocks the current thread until it can enter the SemaphoreSlim while observing a CancellationToken.
4. **Wait(TimeSpan timeout, CancellationToken cancellationToken):** It blocks the current thread until it can enter the SemaphoreSlim, using a TimeSpan that specifies the timeout, while observing a CancellationToken. It returns true if the current thread successfully entered the SemaphoreSlim; otherwise, false.
5. **Wait(int millisecondsTimeout):** It blocks the current thread until it can enter the SemaphoreSlim, using a 32-bit signed integer that specifies the timeout. It returns true if the current thread successfully entered the SemaphoreSlim; otherwise, false.
6. **Wait(int millisecondsTimeout, CancellationToken cancellationToken):** It blocks the current thread until it can enter the SemaphoreSlim, using a 32-bit signed integer that specifies the timeout, while observing a CancellationToken. It returns true if the current thread successfully entered the SemaphoreSlim; otherwise, false.

Parameters:

The following are the parameter descriptions used in the Wait methods.

1. **timeout:** A TimeSpan that represents the number of milliseconds to wait, a TimeSpan that represents -1 milliseconds to wait indefinitely, or a TimeSpan that represents 0 milliseconds to test the wait handle and return immediately.
2. **cancellationToken:** The System.Threading.CancellationToken to observe.
3. **millisecondsTimeout:** The number of milliseconds to wait, System.Threading.Timeout.Infinite(-1) to wait indefinitely, or zero to test the state of the wait handle and return immediately.

Note: The Async versions of all the above methods are also available.

Release Method:

There are two overloaded versions of the Release method available in SemaphoreSlim class. They are as follows:

1. **Release():** It releases the SemaphoreSlim object once. It returns the previous count of the SemaphoreSlim.
2. **Release(int releaseCount):** It releases the SemaphoreSlim object a specified number of times. It returns the previous count of the SemaphoreSlim. Here, the parameter releaseCount specifies the number of times to exit the semaphore.

How SemaphoreSlim work in C#?

When we instantiate a SemaphoreSlim, we can specify the maximum number of threads that can enter the critical section concurrently. We also specify the initial number of threads that can enter the critical section concurrently. This defines the semaphore's count. The count is decremented each time a thread enters the SemaphoreSlim and incremented each time a thread releases the SemaphoreSlim.

To enter the SemaphoreSlim, a thread has to call one of the Wait or WaitAsync overload methods. To release the SemaphoreSlim, the thread has to call one of the Release methods. When the count reaches zero, subsequent calls to the Wait method block until other threads release the SemaphoreSlim. If multiple threads are blocked, there is no guaranteed order, such as FIFO or LIFO, that controls when threads enter the semaphore.

Example to Understand SemaphoreSlim Class in C#:

In the below example, we have created a Function called SemaphoreSlimFunction which gives access to a resource, the Wait method blocks the current thread until it can access the resource, and the Release method is required to release a resource once work is done. To understand SemaphoreSlim, we created five threads inside the Main method which will try to access SemaphoreSlimFunction simultaneously but we limited the access to three threads using the SemaphoreSlim object.

```
using System;
using System.Threading;
namespace SemaphoreSlimDemo
{
    class Program
    {
        //only 3 threads can access resource simlteniously
        static SemaphoreSlim semaphore = new SemaphoreSlim(initialCount:3);
        static void Main(string[] args)
        {
            for (int i = 1; i <= 5; i++)
            {
                int count = i;
                Thread t = new Thread(() => SemaphoreSlimFunction("Thread " +
* count));
                t.Start();
            }
            Console.ReadLine();
        }
        static void SemaphoreSlimFunction(string name, int seconds)
        {
            Console.WriteLine($"{name} Waits to access resource");
            semaphore.Wait();
        }
    }
}
```

```

        Console.WriteLine($"{name} was granted access to resource");
        Thread.Sleep(seconds);
        Console.WriteLine($"{name} is completed");
        semaphore.Release();
    }
}
}

```

Output:

```

Thread 1 Waits to access resource
Thread 1 was granted access to resource
Thread 2 Waits to access resource
Thread 2 was granted access to resource
Thread 3 Waits to access resource
Thread 3 was granted access to resource
Thread 4 Waits to access resource
Thread 5 Waits to access resource
Thread 1 is completed
Thread 4 was granted access to resource
Thread 2 is completed
Thread 5 was granted access to resource
Thread 3 is completed
Thread 4 is completed
Thread 5 is completed

```

Note: We use SemaphoreSlim instance to limit the concurrent threads that can access a shared resource in a multi-threaded environment. If threads trying to access a resource are more than the declared limit, only limited threads will be granted access and others will have to wait.

Another Example to Understand SemaphoreSlim Class in C#:

In the below example, we create one SemaphoreSlim instance with a maximum count of three threads and an initial count of zero threads. The example then starts five tasks, all of which block waiting for the semaphore. The main thread calls the Release(Int32) overload to increase the semaphore count to its maximum, which allows three tasks to enter the semaphore. Each time the semaphore is released, the previous semaphore count is displayed.

```

using System;
using System.Threading;
using System.Threading.Tasks;
public class Example
{
    // Create the semaphore.
    private static SemaphoreSlim semaphore = new SemaphoreSlim(0, 3);
    // A padding interval to make the output more orderly.
    private static int padding;
    public static void Main()

```

```

{
    Console.WriteLine($"{semaphore.CurrentCount} tasks can enter the sema
    Task[] tasks = new Task[5];
    // Create and start five numbered tasks.
    for (int i = 0; i <= 4; i++)
    {
        tasks[i] = Task.Run(() =>
        {
            // Each task begins by requesting the semaphore.
            Console.WriteLine($"Task {Task.CurrentId} begins and waits for
semaphore");
            int semaphoreCount;
            semaphore.Wait();
            try
            {
                Interlocked.Add(ref padding, 100);
                Console.WriteLine($"Task {Task.CurrentId} enters the sema
                // The task just sleeps for 1+ seconds.
                Thread.Sleep(1000 + padding);
            }
            finally
            {
                semaphoreCount = semaphore.Release();
            }
            Console.WriteLine($"Task {Task.CurrentId} releases the semaph
previous count: {semaphoreCount}");
        }));
    }
    // Wait for one second, to allow all the tasks to start and block.
    Thread.Sleep(1000);
    // Restore the semaphore count to its maximum value.
    Console.WriteLine("Main thread calls Release(3) --> ");
    semaphore.Release(3);
    Console.WriteLine($"{semaphore.CurrentCount} tasks can enter the sema
    // Main thread waits for the tasks to complete.
    Task.WaitAll(tasks);
    Console.WriteLine("Main thread Exits");
    Console.ReadKey();
}
}

```

Output:

```
0 tasks can enter the semaphore
Task 1 begins and waits for the semaphore
Task 2 begins and waits for the semaphore
Task 4 begins and waits for the semaphore
Task 3 begins and waits for the semaphore
Task 5 begins and waits for the semaphore
Main thread calls Release(3) --> 3 tasks can enter the semaphore
Task 4 enters the semaphore
Task 3 enters the semaphore
Task 1 enters the semaphore
Task 4 releases the semaphore; previous count: 0
Task 1 releases the semaphore; previous count: 1
Task 2 enters the semaphore
Task 3 releases the semaphore; previous count: 1
Task 5 enters the semaphore
Task 2 releases the semaphore; previous count: 2
Task 5 releases the semaphore; previous count: 1
Main thread Exits
```

In the next article, I am going to discuss [Why and How a Deadlock Occurs in a Multithreaded Application in C#](#) with Examples. Here, in this article, I try to explain **How to Implement Thread Synchronization using SemaphoreSlim Class in C#** with Examples. I hope you enjoy this article and understand the concept of SemaphoreSlim Class in C# with Examples.